第21章 不相交集合：詳解

21-E01 • 集合分割、成功合併與呼叫次數

先把 a 到 f 各建成單點集合。表中的大括號列成員，與代表元素選誰無關。

處理的邊	處理後的集合分割	集合數	累計成功合併
尚無邊	{a},{b},{c},{d},{e},{f}	6	0
ab	{a,b},{c},{d},{e},{f}	5	1
cd	{a,b},{c,d},{e},{f}	4	2
bc	{a,b,c,d},{e},{f}	3	3
ab（重複）	{a,b,c,d},{e},{f}	3	3
ef	{a,b,c,d},{e,f}	2	4
de	{a,b,c,d,e,f}	1	5

六次 UNION，只有五次真正合併。 重複的 ab 仍要找兩個根，但根相同就返回，不能再 LINK。

本題把使用者呼叫的介面計為 m：6 次 MAKE-SET 加 6 次 UNION，故 m=12；UNION 內部共做 12 次 FIND，不再算成額外的外部呼叫。若另以 MAKE／FIND／成功 LINK 展開記帳，則為 6+12+5=23 個基本操作，還須支付每次同根判斷的常數成本。兩種計數方式不可混用。

若同根也加 rank，對單點反覆 UNION(x,x) 就能讓 rank 任意增長，成員卻始終只有一個。這直接違反「根 rank=r 的集合至少有 2^r 個成員」。$n-1$ 限制的是成功合併次數，不是 UNION 呼叫次數。

21-E02 • 尾端接上了，為何還要逐點改指標？

每個成員的 set 指標指向所屬集合紀錄；FIND 會由這份紀錄取得代表。因此接上兩個串列後，被搬動的成員仍須改寫 set。tail 只能省去「走到串列尾端」的成本。

操作	固定把第二串列接到第一尾端：改寫數／代表	短接長：改寫數／代表
UNION(x2,x1)	1／x2	1／x2
UNION(x3,x2)	2／x3	1／x2
UNION(x4,x3)	3／x4	1／x2
UNION(x5,x4)	4／x5	1／x2
UNION(x6,x5)	5／x6	1／x2
總改寫數	15	5

左欄每次由新單點吸收舊長串列，故一般 n 時改寫

$1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}$.

連同 n 次建立，時間為 $\Theta(n^2)$，外部呼叫卻只有 $2n-1$ 次。

右欄第一次同長保留 x2，之後都把新單點接到既有長串列，只改寫 n−1 次，這個序列需 $\Theta(n)$ 時間。但加權合併不保證所有序列都線性；下一題的同大分輪合併會產生 $\Theta(n \log n)$ 次改寫。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

21-E03 • 把總成本拆成每個成員的帳

同長保留第一個集合。每次只列需要改 set 的成員；未列出的成員保留原指標。

合併	被改寫的成員	新集合大小	累計改寫
(a,b)	b	2	1
(c,d)	d	2	2
(e,f)	f	2	3
(g,h)	h	2	4
(a,c)	c,d	4	6
(e,g)	g,h	4	8
(a,e)	e,f,g,h	8	12

所以 a,b,c,d,e,f,g,h 的改寫次數為 0,1,1,2,1,2,2,3，合計 12。h 所屬集合的代表依序為 h→g→e→a；三次改寫後所在集合大小為 2→4→8。

每搬一次，所在集合至少變成兩倍大。 固定任一成員 x；只有它屬於被搬動的較短串列時才收一次費。第 j 次後，所在集合至少有 2^j 個成員；又不超過 n，故 $j \le \lfloor \log_2 n \rfloor$。加總所有成員，得至多 $n\lfloor \log_2 n \rfloor$ 次改寫；各次介面的常數工作另計入 m。

當 $n = 2^k$ 且每輪合併同大集合，第 i 輪有 $n/2^i$ 次合併，每次改寫 2^{i-1} 個成員，所以每輪恰為 n/2，共 k 輪，總數是 $(n/2)\log_2 n$。上界不要求每個人都搬滿 log n 次：本例 a 零次、h 三次。

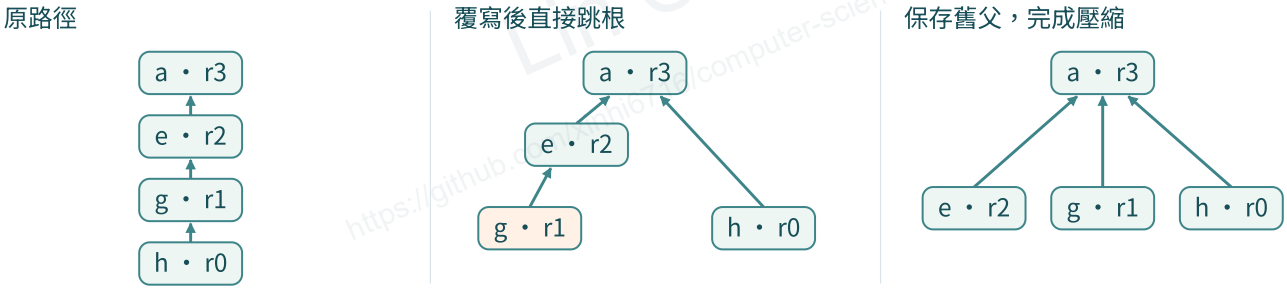
21-E04 • 找到根後，要沿原路逐點改接

依序完成七次 UNION 後，rank 與 parent 如下。根 a 的 parent 是 a 自己。

節點	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
FIND(h) 前 parent	a	a	a	c	a	e	e	g
FIND(h) 後 parent	a	a	a	c	a	e	a	a

第一趟沿 h→g→e→a 找到根，不改指標；第二趟才做下表。**先保存舊父，再覆寫，才能繼續走原來那條路。**

目前 x	先存 old	再設 parent[x]	下一個 x
h	g	a	g
g	e	a	e
e	a	a（原本已是 a）	a
a	—	不改，根自指	停止



若先寫 parent[h]=a，再以 x=parent[h] 繼續，就會直接跳到 a，漏掉 g；e 的連結雖本來就正確，也未依原路走到。此錯誤仍可能回傳正確代表，但沒有完成本題指定的完整路徑壓縮。

d、f 不在本次路徑上，保持 d→c→a、f→e→a。之後 FIND(d) 仍走兩跳，再把 d 接到 a。所有 rank 均不變，不能依目前深度重設：rank 記錄合併歷史，且非根 rank 必須嚴格小於父 rank。

本題同 rank 保留第一根；Cormen 第三版的 LINK 保留第二根。代表名稱與樹形可能不同，但集合分割與 rank 界不變。

21-E05 • Rank 保留合併歷史，不是目前高度

先建四個單點，依序做 UNION(a,b)、UNION(c,d)、UNION(a,c)，同 rank 保留第一根。前兩次使 a、c 的 rank 都升為 1；第三次使 a 的 rank 升為 2，仍有 d→c→a。再做 FIND(d)，b、c、d 全都直接接 a。

此時根 a 的 **rank=2、高度=1**。高度從 2 降為 1，不需要把 rank 一起降為 1。

對「根 rank=r 的集合至少有 2^r 個成員」做操作歸納：

1. MAKE-SET 產生 rank 0 的單點，成員數為 $1 = 2^0$。
2. 不同 rank 合併，保留根的 rank 不變，成員只會增加，下界仍成立。
3. 兩個不同集合的根同為 rank r 時，兩集合各至少有 2^r 人；合併後新根升為 r+1，成員至少 2^{r+1}。
4. FIND 不改成員集合與 rank，所以不影響這個根的下界。

非根的 rank 則固定於它失去根身分之時。那時它的集合曾有至少 2^r 個不同成員；這些成員仍在整個資料結構中，故任一節點都滿足 $2^r \leq n$。**不能把歷史上的集合大小，當成壓縮後這個非根目前的子樹大小。**

因此最大 rank 為 $\lfloor \log_2 n \rfloor$。n=10⁶ 時，$2^{19} = 524288 \leq 10^6 < 1048576 = 2^{20}$，最大為 19。rank 有 0 到 19 共 20 種值，$2^4 < 20 \leq 2^5$，故存 rank 值本身至少需 5 bits；parent 索引和儲存對齊另計。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

21-E06 • 先算門檻，再算每個節點的位能

這裡的函數上標表示重複合成。由 $A_0(j)=j+1$，有 $A_0^{(i)}(j)=j+i$，所以 $A_1(j)=2j+1$。再對 i 歸納：若第 i 次結果為 $2^i(j+1)-1$，代入 $A_1(t)=2t+1$，下一次即 $2^{i+1}(j+1)-1$。因此

$$A_2(j)=2^{j+1}(j+1)-1, \qquad A_3(1)=A_2(A_2(1))=A_2(7)=2047.$$

這不是把同一個數乘 i 次；每一步都把上一個輸出重新當作輸入。

固定 n=16、α=3，原路徑為 h(0)→g(1)→e(2)→p(3)→r(4)，括號是 rank。對非根正 rank 節點，先由父 rank 決定 level，再算該層容許的 iter；位能為 $\phi=(3-\ell)rank-i$。

節點	父 rank：前→後	level：前→後	iter：前→後	位能：前→後
h，rank 0	1→4	不適用	不適用	0→0
g，rank 1	2→4	0→1	1→1	2→1
e，rank 2	3→4	0→0	1→2	5→4
p，rank 3	4→4	0→0	1→1	8→8
r，根 rank 4	4→4	不適用	不適用	12→12

g 的新父 rank=4，跨過 $A_1(1)=3$，未達 $A_2(1)=7$，故 level=1、iter=1。e 則仍在第 0 層：$A_0(2)=3$、$A_0^{(2)}(2)=4$、$A_0^{(3)}(2)=5>4$，故 iter 從 1 升為 2。

路徑位能從 27 降為 25；其餘節點的父與 rank 不變，全森林也減少 2。g、e 各有後方同層非根作見證，是好節點；p 是第 0 層最後的非根，h 的 rank=0，r 是根，三者另計。**同一節點的 level 不下降；iter 只在 level 不變時保證不下降。** 跨層時 iter 可能重設，21-E10 給出實例。

21-E07 • 位能怎麼支付本次真正走過的路？

本題實際成本 c≤2s=14。至少兩個好節點各使 Φ 減少 1，其餘不增加，所以 $\Delta\Phi \le -2$。採固定 $\Psi = 3\Phi$，則

$$\hat{c} = c + \Delta\Psi \le 14 + 3(-2) = 8.$$

一般實際成本至多 cs，取同一個固定常數 C>c，且 $g = \max(0, s - (\alpha + 2))$。若 $s \le \alpha + 2$，不需位能下降，直接有 $\hat{c} \le cs \le c(\alpha + 2)$。若 $s > \alpha + 2$，則

$$\hat{c} \le cs - Cg = (c - C)s + C(\alpha + 2) \le C(\alpha + 2).$$

兩種情況都是 $O(1 + \alpha)$。C 必須在整段分析中固定，不能每次遇到長路徑才臨時增加。

操作	實際成本	Ψ：前→後	ΔΨ	攤銷成本
1	3	0→4	+4	7
2	7	4→2	−2	5
3	2	2→9	+7	9
4	8	9→6	−3	5
合計	20	0→6	+6	26

把 $\hat{c}_i = c_i + \Psi_i - \Psi_{i-1}$ 加總，中間位能互相消去，得實際總成本 $20 = 26 - 6 + 0 \le 26$。這裡可用攤銷總額壓住實際總額，是因為起始位能為 0、末位能非負；不能省略係數，把 $O(s)$ 與 $-\Omega(s - \alpha)$ 直接宣稱相消。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

21-E08 • 一次查找與整段操作，要分開回答

連結／查找規則	可達的單次 FIND 成本	對成本結論的解讀
按 rank 合併，不壓縮	$\Theta(\log n)$	同一條深路徑可被反覆走過
任意不同根相接，不壓縮	$\Theta(n)$	舊根接新單點可造線性鏈
按 rank 合併，加完整壓縮	$\Theta(\log n)$	整段 m 次為 $O(m(1+\alpha(n)))$

取 $n = 2^k$，每輪合併同 rank 的兩個根，共 k 輪；讓某個深葉的集合每輪都被接到另一根底下，可使它深度達 k。建樹時 UNION 只查根，尚未壓縮這個深葉。

不壓縮時，對該葉重複 f 次 FIND 花 $\Theta(f \log n)$，另計建樹 $\Theta(n)$。加入完整壓縮後，第一次仍花 $\Theta(\log n)$；若中間沒有 UNION，後續每次只走一條父邊，f≥1 時總為 $\Theta(\log n + f)$，另計建樹成本。這不表示每一次 FIND 的最壞成本都是常數。

本章定義 $\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}$，而 $A_0(1), A_1(1), A_2(1), A_3(1)$ 依序為 2、3、7、2047。故對 n=0,2,3,7,8,2047,2048，α 依序為 0,0,1,2,3,3,4。

n≤2 時仍有常數操作工作，$O(m\alpha(n))$ 卻會變成零，所以完整定理保留 $1 + \alpha(n)$。n=0 時 FIND／UNION 沒有合法元素可接受；α(0)=0 只是函數定義值。

21-E09 • 結構代表與應用想查的值，可以分開存

最小鍵。 若每次強制小鍵根留下，高 rank 根可能接到低 rank 根，便失去「沿父指標 rank 嚴格增加」的前提。例：先讓鍵 10 的根有 rank=1，再把它接到新單點 key=1、rank=0 底下；此時 1<10，卻是父 rank 比子 rank 小。

保留原本按 rank 的 LINK，在每個根多存 minKey 即可：MAKE-SET(x) 設 minKey[x]=key[x]；合併兩個不同根時，先保存兩邊的 minKey，新根設為兩者最小值。多做一次 min 比較，只增加 O(1) 工作；查詢時先 FIND(x)，再讀新根的 minKey。**O(1) 指附加更新，不是整個查詢的最壞成本。**

有向可達性。 圖只有 a→b 時，忽略方向 UNION(a,b) 會得到同集合，但 b 到不了 a。強連通分量仍是 {a}、{b}；DSU 的同代表只證明忽略方向後連通，不能證明互相可達。

刪邊。 無向圖只有 ab 一條邊，合併後刪掉它，正確結果應拆回兩集合。原 DSU 只提供建立單點、查代表與合併，沒有拆分操作，也未保存足以還原原圖連通性的全部邊資訊。對已存在的 a 再 MAKE-SET 亦違反前提，不能當作合法修補；需採另外的動態連通或離線方法。

21-E10 iter 重設與沿路 level 的反例

32 點：跨層的進度支付 iter 重設

$\alpha(32) = 3$。第一次 FIND(12) 將 rank 2 的節點 12 直接接到 rank 4 的根 0。UNION(16,0) 後，根 16 的 rank 升到 5，因此再次查找的路徑為 $12 \to 0 \to 16$，rank 為 $2, 4, 5$。

點	壓縮前 (ℓ, i, ϕ)	壓縮後 (ℓ, i, ϕ)
12	$(0, 2, 4)$	$(1, 1, 3)$
0	$(0, 1, 11)$	$(0, 1, 11)$
16	根，$\phi = 15$	根，$\phi = 15$

對 12，原父 rank 4 可容許兩次 A_0 迭代，但未達 $A_1(2) = 5$。新父 rank 5 達到第一層，只容許一次 A_1 迭代。位能差是 $-1 \cdot 2 + (2 - 1) = -1$；這次路徑位能由 30 降為 29，其餘節點不變，故全森林也是 $\Delta\Phi = -1$。12 後方有同 level 的非根 0，所以 12 是好節點；0 為該層最後一點，16 為根，均是例外。

16 點：不同節點的 level 可以下降

$\alpha(16) = 3$。類似建立後得到 $2 \to 0 \to 8$，rank 為 $1, 3, 4$。對節點 2，$A_1(1) = 3 \le 3 < A_2(1) = 7$，所以 $(\ell, i, \phi) = (1, 1, 1)$。對節點 0，$A_0(3) = 4 \le 4 < A_1(3) = 7$，所以 $(\ell, i, \phi) = (0, 1, 8)$。根 8 的位能為 12。

沿路 level 是 1、0，沒有同 level 的另一個非根，因此 $G = 0$。FIND(2) 後父 rank 由 3 變 4，但未跨過下一個門檻；三點的位能都不變。不同節點的比較不等於固定節點隨時間的比較，兩者並不衝突。

下界與實際個數

兩例皆為 $s = 3$、$\alpha = 3$，所以 $g = \max(0, 3 - 5) = 0$。第一例 $G = 1 \ge g$，第二例 $G = 0 = g$。短路徑可沒有好節點；$s \le \alpha + 2$ 時，直接以 $cs \le c(\alpha + 2)$ 支付成本，不需要位能下降。

21-E11 不相鄰見證與一次跨兩層

建立 0..31 後，14 的 rank=1、parent=12，12 的 rank=2。FIND(12) 將 12 接到 rank=5 的根 0；UNION(32,0) 將 0 接到 rank=6 的根 32。最後兩個 rank=6 根合併，32 升為 7。查找路徑為 $14 \to 12 \to 0 \to 32$，rank 為 1、2、5、7。

節點	父 rank：前→後	level：前→後	iter：前→後	ϕ：前→後
14	2→7	0→2	1→1	2→0
12	5→7	1→1	1→1	3→3
0	7→7	0→0	2→2	13→13
32（根，rank=7）	7→7	不適用	不適用	21→21

14 壓縮後滿足 $A_2(1) = 7 \le 7 < A_3(1) = 2047$、$A_2^{(2)}(1) = 2047 > 7$，故 level=2、iter=1，位能 $(3-2)1 - 1 = 0$。12 的父 rank 5、7 都在 $A_1(2) = 5$ 與 $A_1^{(2)}(2) = 11$ 之間，故仍為 level=1、iter=1、位能 3。

對 0，$A_0^{(2)}(5) = 7 < A_0^{(3)}(5) = 8$，且 $7 < A_1(5) = 11$，故 level=0、iter=2，位能 $3 \cdot 5 - 2 = 13$。根位能為 21。路徑總和 $39 \to 37$；其餘點的 rank、父 rank 與根身分不變，故全森林 $\Delta\Phi = -2$。

壓縮前 level 為 0、1、0；只有 14 是好節點，見證為 0。12、0 各是其層最後一點，根不參與分層。相鄰比較會漏掉 14。$G = 1$，但 $g = \max(0, 4 - (3+2)) = 0$；實際下降 2 強於保證值 1。

五步為 $7 \ge 7$、$7 \ge A_0(5) = 6$、$6 \ge A_0(2) = 3$、$3 \ge A_0(A_0^{(1)}(1)) = 3$、$3 = A_0^{(2)}(1)$。新父 rank=7 既超過舊層下一個門檻 3，也達 level 2 門檻 7，所以可跨兩層、位能可降超過 1。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.
Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study